CSE 4095-001 — Special Topics in Computer Science and Engineering

Agentic Programming

Fall 2026

Instructor: Wei Wei, Associate Professor in Residence, wei.wei@uconn.edu, ITE 258

Meeting Time: Tuesday and Thursday, 5:00 PM–6:15 PM
Location: MCHU 306
Office Hours: Wednesday, 1:30 PM–2:30 PM

TA: Cheon Park, cheon.park@uconn.edu

TA Office Hours: Wednesday, 3:30 pm – 5:30 pm, ITE 214

Course Description

Generative AI and AI coding agents are changing how software is designed, implemented, tested, and maintained. This course explores an emerging approach to software development in which programmers collaborate with generative AI and agentic coding tools throughout the development process.

The course emphasizes **learning by doing**. Students will use generative AI to explore computational problems, develop interactive visualizations and simulations, create software specifications, build and deploy web applications, and collaborate on a team-based full-stack software project.

The course is organized into three phases:

1. **Explore — Computational Problem Solving with AI**
   Students revisit selected topics in data structures and algorithms through interesting computational problems. Generative AI will be used to explore solutions, generate and analyze code, and create interactive animations, simulations, games, and visualizations.

2. **Engineer — AI-Assisted Software Development**
   Students learn to use AI coding agents and modern development tools to build software systematically. Emphasis will be placed on specification-driven development, task decomposition, version control, debugging, testing, documentation, and deployment.

3. **Create — Team Full-Stack Application**
   Students work in teams to propose, design, implement, test, deploy, and demonstrate a minimum viable product (MVP) consisting of a frontend, backend, and database.

The course focuses not on memorizing the syntax of particular programming languages or AI tools, but on developing the problem-solving, engineering, and critical-thinking skills needed to use AI effectively and responsibly in software development.

Course Learning Outcomes

By the end of this course, students will be able to:

1. **Use generative AI for computational problem solving** to explore problems, develop solution strategies, generate code, and iteratively refine solutions.
2. **Evaluate and verify AI-generated solutions** for correctness, efficiency, robustness, and appropriateness rather than accepting generated results without critical analysis.
3. **Create interactive computational artifacts** such as animations, simulations, visualizations, and games that demonstrate algorithms, computational processes, or problem solutions.
4. **Develop specifications and plans for AI-assisted software projects** by translating application ideas into clear requirements, tasks, and supporting technical documentation.
5. **Use AI coding tools to develop software systematically** through incremental implementation, debugging, testing, version control, documentation, and refinement.
6. **Design, build, and deploy web applications** and explain the roles of frontend interfaces, backend services, APIs, databases, and persistent storage.
7. **Collaborate effectively on a team software project** to define, develop, integrate, test, and deploy a minimum viable product (MVP).
8. **Critically evaluate AI-assisted software development** by recognizing the capabilities and limitations of AI tools and applying human judgment to verify, improve, and communicate AI-assisted work.

Course Philosophy

This course treats generative AI as a **development partner rather than an answer machine**.

Students are encouraged—and expected—to use generative AI extensively. However, students remain responsible for understanding the problems they are solving and for evaluating the correctness and quality of the resulting software.

A recurring development cycle throughout the course will be:

Think → Specify → Build → Verify → Deploy → Reflect → Repeat

The ability to generate code quickly does not eliminate the need for software engineering. Instead, it increases the importance of problem formulation, specification, decomposition, verification, testing, communication, and engineering judgment.

Course Structure

Phase I — Explore

Computational Problem Solving with AI

Weeks 1–5

Students use generative AI to explore computational problems and revisit selected concepts from data structures and algorithms.

Possible topics include:

- recursion
- searching
- sorting
- stacks and queues
- graph traversal
- breadth-first and depth-first search
- state-space exploration
- simulation
- algorithmic complexity

Problems may involve puzzles, games, mathematical experiments, simulations, and other computational challenges.

Students will use AI to help:

- understand problems;
- propose alternative algorithms;
- generate implementations;
- identify and correct errors;
- compare solutions;
- investigate variations of problems; and
- create interactive demonstrations.

A major emphasis will be placed on transforming algorithms into **interactive computational artifacts**.

These may include:

- animations;
- simulations;
- games;
- algorithm visualizations; and
- interactive educational demonstrations.

Stage I web applications will generally be self-contained browser applications requiring **no backend server**.

Phase II — Engineer

AI-Assisted Software Development

Weeks 6–9

Students move from small computational artifacts to structured software development using an AI coding environment or coding agent.

Rather than immediately asking an AI system to build an entire application, students will learn a specification-driven development process.

A typical workflow will be:

Idea → Requirements → Specification → Tasks → Implementation → Testing → Deployment

Depending on the project, documentation may include:

```
SPECIFICATION.md
TODO.md
DATABASE.md
API.md
```

```
UI_GUIDELINES.md
SECURITY.md
DEPLOYMENT.md
README.md
```

Students will learn how these documents provide context for AI coding agents and help maintain consistency throughout a project.

Topics include:

- specification-driven development;
- task decomposition;
- AI coding agents;
- context management;
- Git and GitHub;
- incremental implementation;
- debugging AI-generated software;
- software testing;
- frontend/backend architecture;
- databases and persistent storage;
- deployment; and
- iterative improvement.

Students will individually develop and deploy a small web application during this phase.

Phase III — Create

Team Full-Stack MVP

Weeks 10–14

Students form teams and propose a web application addressing a problem or user need.

Each project must include, where appropriate:

- a frontend user interface;
- a backend service;
- persistent data storage or a database; and
- a deployed application accessible through the Internet.

Teams will define a **minimum viable product (MVP)** and distinguish essential MVP functionality from optional or stretch features.

Each team will develop project documentation that may include:

```
PROJECT_PROPOSAL.md
SPECIFICATION.md
TODO.md
DATABASE.md
API.md
UI_GUIDELINES.md
SECURITY.md
DEPLOYMENT.md
README.md
```

Development will proceed incrementally through project sprints.

Teams will participate in code integration, testing, peer evaluation, user testing, and deployment.

The semester concludes with a final live demonstration and presentation of the deployed MVP.

Tentative Course Schedule

The schedule may be adjusted based on class progress, student interests, and developments in AI programming tools.

Week	Tuesday	Thursday
1	Introduction to Agentic Programming and Vibe Coding	Building the First Interactive Visualization
2	Search, State Spaces, and Computational Puzzles	Visualizing Search Algorithms
3	Recursion, Simulation, and Emergent Behavior	Interactive Simulation Workshop
4	Algorithms as Games and Interactive Experiences	Visualization/Game Development Challenge
5	From Working Code to Good Software	Phase I Project Showcase and Peer Review
6	From Application Idea to Software Specification	Specification and Project Planning Workshop
7	AI Coding Agents and Incremental Development	Individual Application Development Workshop
8	Frontend, Backend, APIs, and Databases	Adding Persistence and Full-Stack Features
9	Debugging, Testing, Git, and	Phase II Application

Week	Tuesday	Thursday
	Deployment	Showcase
10	Team Formation, Project Ideas, and MVP Design	Project Proposals and Team Planning
11	Collaborative Agentic Development and Git Workflow	Sprint 1 — End-to-End Application Development
12	Full-Stack Design, Security, and Integration	Sprint 2 — MVP Development
13	MVP Feature Freeze, Integration, and Testing	Cross-Team User Testing and Refinement
14	Final Project Studio and Presentation Rehearsal	Final MVP Demonstrations

---

Typical Class Format

Because this is a learning-by-doing course, class meetings will generally combine short lectures, demonstrations, discussion, and hands-on development.

A typical Tuesday meeting may consist of:

- 10–20 minutes — introduction of concepts or problems;
- 20–30 minutes — live demonstration and discussion;
- 25–40 minutes — hands-on exploration or development; and
- final minutes — discussion, demonstration, or reflection.

Thursday meetings will generally be more workshop-oriented:

- short mini-lecture or demonstration;
- extended development/workshop period;
- instructor and peer consultation; and
- progress demonstrations or reflection.

As the semester progresses into the team-project phase, a larger portion of class time will be devoted to project development.

Tentative Grading

Component	Weight

Component	Weight
Phase I — Computational Exploration Assignments	25%
Phase I — Interactive Visualization/Game Project	10%
Phase II — Individual Deployed Web Application	25%
Phase III — Team Full-Stack MVP	30%
Participation, Peer Review, and Reflection	10%
Total	**100%**

The exact number and weighting of assignments may be adjusted during the semester.

Phase III Team Project Evaluation

The team project will be evaluated on both the quality of the final product and individual contributions.

Evaluation may consider:

- problem definition and MVP scope;
- application functionality;
- frontend usability;
- backend and database design;
- reliability and testing;
- deployment;
- project documentation;
- effective use of AI-assisted development;
- team collaboration;
- individual contributions;
- final demonstration and presentation; and
- reflection on the development process.

Students should not assume that all members of a team will automatically receive the same project grade. Individual contribution and peer evaluation may be considered when assigning grades.

Use of Generative AI

Generative AI is an integral part of this course.

Students are **expected and encouraged** to use tools such as ChatGPT and AI-assisted programming environments when completing course activities and projects.

Permitted and encouraged uses include:

- exploring problems;
- brainstorming solutions;
- explaining algorithms;
- generating code;
- generating tests;
- debugging;
- designing user interfaces;
- creating project specifications;
- generating documentation;
- reviewing code;
- refactoring;
- analyzing errors; and
- exploring alternative implementations.

However, using AI does not transfer responsibility for the work to the AI system.

Students are responsible for:

1. understanding the problem being addressed;
2. evaluating the proposed solution;
3. testing generated software;
4. identifying and correcting errors;
5. understanding the major architectural decisions in their projects;
6. protecting sensitive information and credentials; and
7. being able to explain and demonstrate submitted work.

Students may be asked to explain any part of their submitted project or development process.

Being unable to explain significant portions of submitted work may affect the evaluation of the assignment or project.

Verification of AI-Generated Content

Generative AI systems can produce:

- incorrect code;

- nonexistent APIs;
- insecure implementations;
- inefficient algorithms;
- misleading explanations;
- unnecessary complexity; and
- solutions that appear correct but fail on important edge cases.

Students should therefore treat AI-generated content as a **proposal to be evaluated**, not as an authoritative solution.

Testing, experimentation, code review, and human judgment are essential components of the development process and are learning objectives of this course.

Version Control and Project Management

Students will use Git and GitHub or comparable version-control tools for software projects.

Students are expected to commit their work regularly.

Team projects should maintain a meaningful development history demonstrating incremental progress and contributions from team members.

Projects may also use:

- issues;
- milestones;
- project boards;
- branches; and
- pull requests

to organize collaborative development.

Deployment

An important objective of the course is moving software beyond the development computer.

When required, projects should be deployed so that they can be accessed through a web browser.

Students will gain experience with appropriate deployment platforms and will be responsible for verifying that deployed applications function correctly.

A project that functions only on a student's development computer may not satisfy deployment requirements.

Project Documentation

Software documentation serves two audiences in this course:

1. **Human developers**, who need to understand the project; and
2. **AI coding agents**, which require accurate context to work effectively.

Students are therefore expected to maintain project documentation as the implementation evolves.

Specifications and TODO files should not simply be generated at the beginning of a project and forgotten. They should be treated as living documents and updated when important project decisions change.

Collaboration

Discussion and collaboration are encouraged.

During individual assignments, students may discuss problems, algorithms, development approaches, debugging strategies, and AI workflows with classmates. Unless explicitly designated as a group assignment, however, each student must develop and submit their own project.

During the team-project phase, collaboration within the assigned team is expected.

Students should clearly communicate responsibilities, maintain shared project documentation, and contribute meaningfully to the team's development process.

Learning by Doing

This course places substantial emphasis on experimentation and project development during class.

Students should therefore bring a laptop capable of running the required development tools to each class meeting.

Regular participation in workshops, demonstrations, peer reviews, and team-development activities is an important component of the course.

Because many activities involve active development and collaboration, some learning experiences may be difficult to reproduce simply by reviewing lecture materials afterward.

Software and Tools

The course may use tools such as:

- ChatGPT;
- Cursor, Codex, or comparable AI coding tools;
- Visual Studio Code or another code editor;
- Git;
- GitHub;
- modern web browsers;
- Python;
- HTML, CSS, and JavaScript;
- frontend and backend web frameworks;
- database systems; and
- cloud deployment platforms.

Specific tools may change during the semester as AI development technologies evolve.

The course emphasizes transferable development practices rather than dependence on a particular commercial AI product.

Course Materials

There is no required textbook currently planned for the course.

Course materials may include:

- instructor notes;
- problem descriptions;
- software documentation;
- online technical resources;

- AI-generated examples;
- research or industry articles; and
- example software projects.

Additional resources will be announced during the semester.

Tentative Nature of the Syllabus

Agentic programming and AI-assisted software development are rapidly evolving areas. New tools and capabilities may emerge during the semester.

Consequently, the instructor may modify topics, tools, assignments, or the course schedule when doing so provides a better learning experience.

Significant changes will be communicated to students in advance.